

# Looker Developer Guide

## Power BI -> LookML build playbook for developers

Source PBIX: movie\_rental\_analysis.pbix

LookML model: movie\_rental\_analysis

Primary fact / explore: film

Approach: deterministic Phase 2 (inventory-driven)

This PDF: full instructions, checks, suggestions, examples, and gaps

### Inside this guide

- \* Section 4: every PBI object type with step-by-step build instructions (one topic per page)
- \* Section 5: concrete build plan for THIS PBIX (views, joins, measures, M)
- \* Section 6: prioritized GAP list with actions
- \* Sections 7-8: validation checklist and acceptance criteria

# 1. How to use this guide

- \* Read Best practices first - mandatory.
- \* Complete Prerequisites before editing LookML.
- \* Read each Object equivalence page fully (Build + Check + Suggestions + Example).
- \* Execute Build plan for this PBIX.
- \* Close HIGH gaps (or get written sign-off).
- \* Sign off using Validation and acceptance.

## 2. Best practices (mandatory)

### Primary key on every view

Put primary\_key: yes on the unique key. Required for symmetric aggregates.

### Explicit join relationship

Always set relationship: many\_to\_one (or the correct cardinality). Never omit it.

### Measures reference dimensions

Prefer sql: \${dimension} over raw \${TABLE}.col when the dimension exists.

### Warehouse owns ETL

Power Query M is NOT LookML. Rebuild transforms in dbt/Dataform/SQL first.

### Granular includes

Prefer include per view file (looker-skills modeling guidelines).

### Explore descriptions

Every explore needs label + description.

### No silent M:M / inactive joins

Review fan-out risk; use bridge tables or aliased joins intentionally.

### No KPI claims without tests

Complex DAX TODOs are unfinished until side-by-side parity passes.

### 3. Prerequisites checklist

- \* ☐ Looker project + Git branch ready
- \* ☐ Admin connection created
- \* ☐ Warehouse tables exist (or creation tickets filed)
- \* ☐ Phase 1 inventory is for THIS PBIX
- \* ☐ LOOKML\_PROJECT.zip imported
- \* ☐ SQL dialect known (BigQuery SAFE\_DIVIDE vs Snowflake DIV0)

Do not start LookML polish until connection and warehouse tables are real. Placeholder  
YOUR\_LOOKER\_CONNECTION / YOUR\_PROJECT.YOUR\_DATASET will fail validation.

## 4.1 Object equivalence

### Table (business) -> view (.view.lkml)

#### What it means

One business table becomes one LookML view file pointed at a warehouse table.

#### How to build (follow these steps)

- \* 1. Create file views/<snake\_case\_table>.view.lkml.
- \* 2. Declare view: <name> with label matching the Power BI table business name.
- \* 3. Set sql\_table\_name to the real warehouse table (replace YOUR\_PROJECT.YOUR\_DATASET).
- \* 4. Add dimensions for all columns that analysts need.
- \* 5. Put primary\_key: yes on the unique key dimension (first field).

#### What to check

- \* [ ] View compiles in Looker Validator with 0 errors.
- \* [ ] sql\_table\_name resolves on the connection (table exists).
- \* [ ] Exactly one primary\_key: yes in the view.
- \* [ ] Row count in Explore (count) is plausible vs Power BI table rows.

#### Suggestions / best practice

- \* Prefer clear warehouse names (restaurants, not Sheet1) while keeping friendly labels.
- \* Do not model LocalDateTable\_\* / DataTableTemplate\_\* as views.
- \* Hide technical surrogate keys with hidden: yes if users should not see them.

#### Example LookML

```
view: sheet1 {  
  sql_table_name: `proj.dataset.sheet1` ;;  
  dimension: restaurant_id {  
    primary_key: yes  
    type: number  
    sql: ${TABLE}.RestaurantID ;;  
  }  
}
```

#### References

- \* <https://cloud.google.com/looker/docs/reference/param-view-view>
- \* <https://github.com/looker-open-source/looker-skills/blob/main/skills/lookml-view/SKILL.md>

## 4.2 Object equivalence

### Column -> dimension (or dimension\_group for dates)

#### What it means

Physical columns become dimensions; dates become dimension\_group timeframes.

#### How to build (follow these steps)

- \* 1. Map each inventory column to a snake\_case LookML field name.
- \* 2. Choose type: string, number, yesno, or time (dimension\_group).
- \* 3. For dates, use dimension\_group: <name> { type: time timeframes: [...] sql: ... ;; }.
- \* 4. Copy business meaning into label: and description:.
- \* 5. Preserve format intent (currency, percent) via value\_format\_name when useful.

#### What to check

- \* [ ] Field appears in Explore field picker with correct label.
- \* [ ] Filtering/grouping on the field returns values matching Power BI.
- \* [ ] Date fields support month/quarter/year without extra DAX date tables.

#### Suggestions / best practice

- \* Never expose auto-date table columns; use business dates + timeframes.
- \* Use group\_label to cluster related fields (IDs, geography, flags).
- \* If column names have spaces, quote in SQL: \${TABLE}.`Country name`.

#### Example LookML

```
dimension_group: order {  
  type: time  
  timeframes: [raw, date, week, month, quarter, year]  
  sql: ${TABLE}.OrderDate ;;  
  datatype: date  
}
```

#### References

- \* <https://cloud.google.com/looker/docs/reference/param-field-dimension>
- \* <https://cloud.google.com/looker/docs/reference/param-field-dimension-group>

## 4.3 Object equivalence

### Measure (DAX) -> measure

#### What it means

DAX measures become LookML measures using sum/average/count\_distinct patterns; measure-of-measures use type: number + \${measure}; ratios use NULLIF; dependent measures are listed in MEASURE\_DEPENDENCIES.md.

#### How to build (follow these steps)

- \* 1. Classify each DAX expression (simple aggregate vs measure-math vs CALCULATE vs time intelligence).
- \* 2. Implement simple aggregates with type: sum / average / count\_distinct and \${dimension} sql.
- \* 3. Implement ratios with  $1.0 * \text{\${num}} / \text{NULLIF}(\text{\${den}}, 0)$  and value\_format\_name (official Looker division pattern).
- \* 4. For CALCULATE with simple equality/ISBLANK filters, use filters: { field: "value" } on a base aggregate.
- \* 5. For DATESYTD/DATEADD/SUMX/etc., leave TODO and implement with Looker period analysis or warehouse metrics.
- \* 6. When a measure references other measures, emit type: number, list DEPENDS ON, and order fields so bases come first.
- \* 7. Keep original DAX in description: until KPI parity is signed off.

#### What to check

- \* [ ] Measure returns non-null for a known filter set.
- \* [ ] Side-by-side vs Power BI for at least 3 filter combinations.
- \* [ ] No fan-out inflation after joins (compare count vs count\_distinct on PK).
- \* [ ] Dependent measures compile only after their bases exist (Looker validator).

#### Suggestions / best practice

- \* Prefer \${dimension} references inside aggregate measure sql (looker-skills).
- \* Never put filters: on type: number - filter the composing aggregates instead.
- \* Do not claim parity for COMPLEX DAX without tests.
- \* If PBIX has 0 measures, still add explicit KPIs users expect (counts, averages).
- \* Open MEASURE\_DEPENDENCIES.md for this PBIX before editing KPI fields.

#### Example LookML

```
measure: total_sales {
  type: sum
  sql: ${sales_amount} ;;
  value_format_name: usd_0
}
measure: avg_rating {
  type: average
  sql: ${rating} ;;
}
measure: seps_yoy_var {
  type: number
  description: "DEPENDS ON: Seps, Seps SPLY"
  sql: ${seps} - ${seps_sply} ;;
}
measure: to_percent {
  type: number
  sql:  $1.0 * \text{\${seps}} / \text{NULLIF}(\text{\${actives}}, 0)$  ;;
  value_format_name: percent_2
}
```

#### References

- \* <https://cloud.google.com/looker/docs/reference/param-field-measure>
- \* <https://cloud.google.com/looker/docs/reference/param-measure-types>
- \* <https://cloud.google.com/looker/docs/reference/param-field-filters>
- \* <https://cloud.google.com/looker/docs/best-practices/how-to-troubleshoot-fields-with-division-displ>
- \* aying-0

\* <https://cloud.google.com/looker/docs/reference/field-reference>

<https://github.com/looker-open-source/looker-skills/blob/main/skills/lookml-modeling-guidelines/SK>

\* ILL.md

## 4.4 Object equivalence

### Relationship (M:1 From->To) -> explore join (relationship: many\_to\_one)

#### What it means

Power BI relationships become Explore joins with explicit relationship and sql\_on.

#### How to build (follow these steps)

- \* 1. Choose the fact table as the explore base (usually the many side).
- \* 2. For each active M:1 relationship, add a join to the one-side view.
- \* 3. Set type: left\_outer (typical) and relationship: many\_to\_one.
- \* 4. Write sql\_on using \${view.field} syntax on both sides.
- \* 5. If the same dim joins twice, use from: + a unique join name (alias).
- \* 6. For joins off an intermediate dim, add required\_joins: [intermediate\_view].

#### What to check

- \* [ ] Join keys match Phase 1 inventory from/to columns exactly.
- \* [ ] Exploring fact+dim does not multiply fact rows unexpectedly.
- \* [ ] Inactive Power BI relationships are not used as the default path.

#### Suggestions / best practice

- \* M:M is a HIGH risk gap - prefer a warehouse bridge table.
- \* Looker has no inactive join flag; keep inactive paths aliased and hidden until needed.
- \* Always declare relationship: (looker-skills requirement).

#### Example LookML

```
explore: sheet1 {  
  join: sheet2 {  
    type: left_outer  
    relationship: many_to_one  
    sql_on: ${sheet1.country_code} = ${sheet2.country_id} ;;  
  }  
}
```

#### References

- \* <https://cloud.google.com/looker/docs/reference/param-explore-join>
- \* <https://github.com/looker-open-source/looker-skills/blob/main/skills/lookml-explore/SKILL.md>

## 4.5 Object equivalence

### Calculated column (DAX) -> dimension (prefer warehouse column)

#### What it means

Row-level DAX calculated columns should be materialized in the warehouse, then exposed as dimensions.

#### How to build (follow these steps)

- \* 1. List every business calculated column from Phase 1 (02\_dax\_objects.json).
- \* 2. Recreate the expression in warehouse SQL (CASE/TRIM/date logic).
- \* 3. Add the physical column to the warehouse table.
- \* 4. Expose it in the view as a dimension with description noting original DAX.
- \* 5. Only use LookML-only sql: for trivial expressions you accept maintaining in Looker.

#### What to check

- \* [ ] Warehouse column exists and is populated for sample keys.
- \* [ ] Dimension values match Power BI calculated column for sample rows.

#### Suggestions / best practice

- \* Warehouse-first keeps LookML thin and testable.
- \* Financial month / rating buckets / TRIM fields are typical warehouse CASE statements.
- \* Do not leave DAX in production LookML descriptions without an owner.

#### Example LookML

```
dimension: cuisiness {  
  description: "PBI calc: TRIM(Sheet1[Cuisines]) - prefer warehouse"  
  type: string  
  sql: ${TABLE}.cuisiness ;;  
}
```

#### References

- \* <https://cloud.google.com/looker/docs/lookml-terms-and-concepts>

## 4.6 Object equivalence

### Calculated table -> view (sql\_table\_name or derived\_table)

#### What it means

DAX calculated tables become warehouse tables (preferred) or LookML derived tables.

#### How to build (follow these steps)

- \* 1. Identify calculated tables in inventory (is\_calculated\_table).
- \* 2. Decide: materialize in warehouse vs SQL derived table in LookML.
- \* 3. Prefer warehouse for anything used in multiple explores or large grain.
- \* 4. Create the view with primary\_key and needed dimensions.

#### What to check

- \* [ ] Table grain and keys documented.
- \* [ ] Joins to the calculated table match Power BI relationships.

#### Suggestions / best practice

- \* Avoid embedding heavy DAX table logic only inside LookML.
- \* Document why the calculated table exists (seed, bridge, snapshot).

#### Example LookML

```
view: adjustment_factor_pct {  
  sql_table_name: `proj.dataset.adjustment_factor_pct` ;;  
  dimension: adjustment_factor_pct {  
    primary_key: yes  
    type: number  
    sql: ${TABLE}.`Adjustment factor (%)` ;;  
  }  
}
```

#### References

- \* <https://github.com/looker-open-source/looker-skills/blob/main/skills/lookml-view/SKILL.md>

## 4.7 Object equivalence

### Power Query M -> Warehouse table/view + straight LookML view (preferred); SDT only as temporary bridge

#### What it means

Each M query gets a deterministic recommendation: warehouse straight view, warehouse transform, warehouse seed, or temporary LookML SQL derived table.

#### How to build (follow these steps)

- \* 1. Open lookml/m\_migration/M\_QUERY\_RECOMMENDATIONS.md for this PBIX.
- \* 2. For each query, follow the recommended pattern (usually warehouse table + straight view).
- \* 3. Implement the matching sql/<query>.sql stub in your warehouse/dbt project.
- \* 4. Apply lookml\_stubs/<query>\_recommended.lkml guidance; update views/<query>.view.lkml sql\_table\_name.
- \* 5. Use SDT only when the recommendation explicitly allows a temporary bridge.
- \* 6. Do not re-implement File.Contents / Excel paths inside Looker.

#### What to check

- \* [ ] Every M query has a recommendation row and SQL stub in the ZIP
- \* [ ] Warehouse row counts match Power BI for sample queries
- \* [ ] Generated views resolve after sql\_table\_name is updated
- \* [ ] No heavy merge/append logic left only in LookML

#### Suggestions / best practice

- \* Best practice order: warehouse table/view > temporary SDT > avoid NDT as M replacement.
- \* File CSV M queries -> load to warehouse, then straight view.
- \* SQL-in-M -> warehouse view with that SELECT, or temporary SDT.
- \* Merge/append/heavy M -> dbt/Dataform model, never LookML-only.
- \* Embedded #table -> warehouse seed (+ tiny SDT only if temporary).

#### Example LookML

```
-- warehouse
CREATE OR REPLACE TABLE `proj.dataset.actor` AS SELECT ...;

# LookML straight view
view: actor {
  sql_table_name: `proj.dataset.actor` ;;
}
```

#### References

- \* <https://cloud.google.com/looker/docs/lookml-terms-and-concepts>
- \* <https://github.com/looker-open-source/looker-skills/blob/main/skills/lookml-view/SKILL.md>

## 4.8 Object equivalence

### Hierarchy -> drill\_fields / sets / dimension\_group timeframes

#### What it means

Hierarchies become drill paths or timeframes - not separate LookML hierarchy objects.

#### How to build (follow these steps)

- \* 1. For date hierarchies, rely on dimension\_group timeframes (year > quarter > month > date).
- \* 2. For attribute hierarchies, add drill\_fields: [level1, level2, level3] on the top field.
- \* 3. Optionally create a set: for curated field lists.

#### What to check

- \* [ ] Users can drill year to month to date in Explores.
- \* [ ] No dependency on LocalDateTable\_\* hierarchies.

#### Suggestions / best practice

- \* Skip migrating auto-date hierarchies from internal tables.
- \* Use drill\_fields on measures for guided analysis paths.

#### Example LookML

```
dimension: country {  
  type: string  
  sql: ${TABLE}.Country ;;  
  drill_fields: [city, locality, restaurant_name]  
}
```

#### References

- \* <https://cloud.google.com/looker/docs/reference/param-field-drill-fields>

## 4.9 Object equivalence

### RLS / OLS -> access\_grant / access\_filter / required\_access\_grants

#### What it means

Power BI row/object security maps to Looker access filters and grants - not automatic.

#### How to build (follow these steps)

- \* 1. Read Phase 1 RLS inventory (empty list means none found).
- \* 2. Define Looker user attributes (e.g. country, org\_id).
- \* 3. Add access\_filter on the explore or required\_access\_grants on sensitive fields.
- \* 4. Test with users in and out of each role.

#### What to check

- \* [ ] Unauthorized users cannot see restricted rows/fields.
- \* [ ] Authorized users still see full expected grain.

#### Suggestions / best practice

- \* Even if RLS is empty, confirm with security owners before production.
- \* Prefer access\_filter for row security; access\_grant for field-level.

#### Example LookML

```
explore: sheet1 {  
  access_filter: {  
    field: sheet2.country_name  
    user_attribute: allowed_country  
  }  
}
```

#### References

\*

<https://github.com/looker-open-source/looker-skills/blob/main/skills/lookml-access-grants/SKILL.md>

## 4.10 Object equivalence

### LocalDateTable\_\* / DataTableTemplate\_\* -> Skip (use business date + dimension\_group)

#### What it means

Internal auto-date tables are Power BI implementation detail - do not migrate.

#### How to build (follow these steps)

- \* 1. Identify internal tables in inventory and exclude them from LookML views.
- \* 2. Find the real business date column on fact/dim tables.
- \* 3. Model that column as dimension\_group.

#### What to check

- \* [ ] No LocalDateTable\_\* views in the LookML project.
- \* [ ] Time-based analysis works via business dates.

#### Suggestions / best practice

- \* Mark this as an intentional skip in gap docs (severity LOW).

#### Example LookML

```
# SKIP creating view: local_date_table_...  
# USE dimension_group on FactTable.ServiceDate instead
```

#### References

- \* <https://cloud.google.com/looker/docs/reference/param-field-dimension-group>

## 4.11 Object equivalence

### Model / Dataset -> model (.model.lkml) + connection

#### What it means

The Power BI dataset becomes a LookML model file with connection, includes, explores.

#### How to build (follow these steps)

- \* 1. Open models/<pbix\_stem>.model.lkml from LOOKML\_PROJECT.zip.
- \* 2. Set connection: to the Looker Admin connection name.
- \* 3. Keep include: lines for each view (granular includes preferred).
- \* 4. Confirm the primary explore points at the fact view.
- \* 5. Configure datagroup / persist\_with when ETL cadence is known.

#### What to check

- \* [ ] Model appears in Looker and Explore loads.
- \* [ ] Includes resolve; no missing view errors.

#### Suggestions / best practice

- \* Hide dimension-only QA explores from end users (hidden: yes).
- \* One focused explore beats a mega-explore when possible.

#### Example LookML

```
connection: "warehouse_bq"
include: "/views/sheet1.view.lkml"
include: "/views/sheet2.view.lkml"
explore: sheet1 {
  label: "Restaurants"
  description: "Migrated from Power BI Zomato model."
}
```

#### References

- \* <https://cloud.google.com/looker/docs/lookml-terms-and-concepts>
- <https://github.com/looker-open-source/looker-skills/blob/main/skills/lookml-modeling-guidelines/SK>
- \* ILL.md

## 5. Build plan for this PBIX

### Step A - Import LookML ZIP

- \* Unzip LOOKML\_PROJECT.zip into the Looker project.
- \* Open models/<model>.model.lkml and set connection.
- \* Run LookML Validator (expect table errors until Step B).

### Step B - Point each view at the warehouse

For each view: set sql\_table\_name, confirm PK and columns exist.

#### View `actor` (Power BI: actor)

File: views/actor.view.lkml | columns: 4 | PK guess: actor\_id | date cols: last\_update

- \* Confirm primary\_key matches warehouse uniqueness.
- \* Map dates to dimension\_group timeframes.

#### View `address` (Power BI: address)

File: views/address.view.lkml | columns: 9 | PK guess: address\_id | date cols: last\_update

- \* Confirm primary\_key matches warehouse uniqueness.
- \* Map dates to dimension\_group timeframes.

#### View `category` (Power BI: category)

File: views/category.view.lkml | columns: 3 | PK guess: category\_id | date cols: last\_update

- \* Confirm primary\_key matches warehouse uniqueness.
- \* Map dates to dimension\_group timeframes.

#### View `city` (Power BI: city)

File: views/city.view.lkml | columns: 4 | PK guess: city\_id | date cols: last\_update

- \* Confirm primary\_key matches warehouse uniqueness.
- \* Map dates to dimension\_group timeframes.

#### View `country` (Power BI: country)

File: views/country.view.lkml | columns: 3 | PK guess: country\_id | date cols: last\_update

- \* Confirm primary\_key matches warehouse uniqueness.
- \* Map dates to dimension\_group timeframes.

#### View `customer` (Power BI: customer)

File: views/customer.view.lkml | columns: 10 | PK guess: customer\_id | date cols: create\_date, last\_update

- \* Confirm primary\_key matches warehouse uniqueness.
- \* Map dates to dimension\_group timeframes.

#### View `film` (Power BI: film)

File: views/film.view.lkml | columns: 13 | PK guess: film\_id | date cols: last\_update

- \* Confirm primary\_key matches warehouse uniqueness.
- \* Map dates to dimension\_group timeframes.

#### View `film\_actor` (Power BI: film\_actor)

File: views/film\_actor.view.lkml | columns: 3 | PK guess: actor\_id | date cols: last\_update

- \* Confirm primary\_key matches warehouse uniqueness.
- \* Map dates to dimension\_group timeframes.

#### View `film\_category` (Power BI: film\_category)

File: views/film\_category.view.lkml | columns: 3 | PK guess: film\_id | date cols: last\_update

- \* Confirm primary\_key matches warehouse uniqueness.
- \* Map dates to dimension\_group timeframes.

#### View `film\_text` (Power BI: film\_text)

File: views/film\_text.view.lkml | columns: 3 | PK guess: film\_id | date cols: none

- \* Confirm primary\_key matches warehouse uniqueness.
- \* Map dates to dimension\_group timeframes.

#### View `inventory` (Power BI: inventory)

File: views/inventory.view.lkml | columns: 4 | PK guess: inventory\_id | date cols: last\_update

- \* Confirm primary\_key matches warehouse uniqueness.
- \* Map dates to dimension\_group timeframes.

### View `language` (Power BI: language)

File: views/language.view.lkml | columns: 3 | PK guess: language\_id | date cols: last\_update

\* Confirm primary\_key matches warehouse uniqueness.

\* Map dates to dimension\_group timeframes.

### View `payment` (Power BI: payment)

File: views/payment.view.lkml | columns: 7 | PK guess: payment\_id | date cols: payment\_date, last\_update

\* Confirm primary\_key matches warehouse uniqueness.

\* Map dates to dimension\_group timeframes.

### View `rentat` (Power BI: rentat)

File: views/rentat.view.lkml | columns: 7 | PK guess: rental\_id | date cols: rental\_date, return\_date, last\_update

\* Confirm primary\_key matches warehouse uniqueness.

\* Map dates to dimension\_group timeframes.

### View `staff` (Power BI: staff)

File: views/staff.view.lkml | columns: 11 | PK guess: staff\_id | date cols: last\_update

\* Confirm primary\_key matches warehouse uniqueness.

\* Map dates to dimension\_group timeframes.

### View `store` (Power BI: store)

File: views/store.view.lkml | columns: 4 | PK guess: store\_id | date cols: last\_update

\* Confirm primary\_key matches warehouse uniqueness.

\* Map dates to dimension\_group timeframes.

## Step C - Explores and joins

Primary explore base: film. Set type + relationship + sql\_on on every join.

**film\_actor[actor\_id] -> actor[actor\_id] [M:1, ACTIVE]**

Looker action: join actor relationship from PBI M:1

**address[city\_id] -> city[city\_id] [M:1, ACTIVE]**

Looker action: join city relationship from PBI M:1

**city[country\_id] -> country[country\_id] [M:1, ACTIVE]**

Looker action: join country relationship from PBI M:1

**film[language\_id] -> language[language\_id] [M:1, ACTIVE]**

Looker action: join language relationship from PBI M:1

**film\_category[category\_id] -> category[category\_id] [M:1, ACTIVE]**

Looker action: join category relationship from PBI M:1

**film[film\_id] -> film\_category[film\_id] [M:1, ACTIVE]**

Looker action: join film\_category relationship from PBI M:1

**film[film\_id] -> film\_text[film\_id] [M:1, ACTIVE]**

Looker action: join film\_text relationship from PBI M:1

**store[store\_id] -> staff[store\_id] [M:1, ACTIVE]**

Looker action: join staff relationship from PBI M:1

**film\_actor[film\_id] -> film[film\_id] [M:1, ACTIVE]**

Looker action: join film relationship from PBI M:1

**customer[address\_id] -> address[address\_id] [M:1, ACTIVE]**

Looker action: join address relationship from PBI M:1

**payment[customer\_id] -> customer[customer\_id] [M:1, ACTIVE]**

Looker action: join customer relationship from PBI M:1

**customer[store\_id] -> store[store\_id] [M:1, ACTIVE]**

Looker action: join store relationship from PBI M:1

**rentat[inventory\_id] -> inventory[inventory\_id] [M:1, ACTIVE]**

Looker action: join inventory relationship from PBI M:1

**rentat[rental\_id] -> payment[rental\_id] [M:1, ACTIVE]**

Looker action: join payment relationship from PBI M:1

**rentat[staff\_id] -> staff[staff\_id] [M:1, INACTIVE]**

Looker action: join staff relationship from PBI M:1

\* Keep aliased; do not use as default path until a measure needs it.

**inventory[film\_id] -> film[film\_id] [M:1, ACTIVE]**

Looker action: join film relationship from PBI M:1

## Step D - Measures

Mapped measures: 5. TODOs: 2.

- \* [OK] payment.Revenue - strategy=direct\_sum
- \* [TODO] inventory.Total Films Rented - strategy=complex\_todo
- \* [OK] inventory.Average Inventory Value - strategy=direct\_average
- \* [OK] inventory.Inventory Turnover Rate - strategy=measure\_ratio
- \* [TODO] actor.FilmPopularity - strategy=complex\_todo

## Step E - Calculated columns (build in warehouse)

1 calculated columns - materialize then expose as dimensions.

- \* customer.Employment Duration: DATEDIFF('customer'[create\_date], TODAY(), YEAR)

## Step F - Power Query M (Looker / warehouse equivalent)

Best practice: warehouse table/view + straight LookML view. SDT only as temporary bridge. Heavy M stays in ETL. Full stubs are in LOOKML\_PROJECT.zip under lookml/m\_migration/.

### actor -> warehouse\_table\_plus\_straight\_view

Looker object: straight view (sql\_table\_name) over warehouse table

Why: M is a file extract with light Promote Headers / Changed Type. Looker best practice: load to warehouse, then use a normal view - not a derived table.

- \* 1. Land the CSV/Excel in cloud storage or ingest to a staging table.
- \* 2. Create curated table `YOUR\_PROJECT.YOUR\_DATASET.actor` with casts matching M Changed Type.
- \* 3. Point generated LookML views/actor.view.lkml sql\_table\_name at that table.
- \* 4. Do not use File.Contents paths from M in Looker.
- \* Files: m\_migration/sql/actor.sql and lookml\_stubs/actor\_recommended.lkml

### address -> warehouse\_table\_plus\_straight\_view

Looker object: straight view (sql\_table\_name) over warehouse table

Why: M is a file extract with light Promote Headers / Changed Type. Looker best practice: load to warehouse, then use a normal view - not a derived table.

- \* 1. Land the CSV/Excel in cloud storage or ingest to a staging table.
- \* 2. Create curated table `YOUR\_PROJECT.YOUR\_DATASET.address` with casts matching M Changed Type.
- \* 3. Point generated LookML views/address.view.lkml sql\_table\_name at that table.
- \* 4. Do not use File.Contents paths from M in Looker.
- \* Files: m\_migration/sql/address.sql and lookml\_stubs/address\_recommended.lkml

### city -> warehouse\_table\_plus\_straight\_view

Looker object: straight view (sql\_table\_name) over warehouse table

Why: M is a file extract with light Promote Headers / Changed Type. Looker best practice: load to warehouse, then use a normal view - not a derived table.

- \* 1. Land the CSV/Excel in cloud storage or ingest to a staging table.
- \* 2. Create curated table `YOUR\_PROJECT.YOUR\_DATASET.city` with casts matching M Changed Type.
- \* 3. Point generated LookML views/city.view.lkml sql\_table\_name at that table.
- \* 4. Do not use File.Contents paths from M in Looker.
- \* Files: m\_migration/sql/city.sql and lookml\_stubs/city\_recommended.lkml

### country -> warehouse\_table\_plus\_straight\_view

Looker object: straight view (sql\_table\_name) over warehouse table

Why: M is a file extract with light Promote Headers / Changed Type. Looker best practice: load to warehouse, then use a normal view - not a derived table.

- \* 1. Land the CSV/Excel in cloud storage or ingest to a staging table.
- \* 2. Create curated table `YOUR\_PROJECT.YOUR\_DATASET.country` with casts matching M Changed Type.
- \* 3. Point generated LookML views/country.view.lkml sql\_table\_name at that table.
- \* 4. Do not use File.Contents paths from M in Looker.
- \* Files: m\_migration/sql/country.sql and lookml\_stubs/country\_recommended.lkml

### **category -> warehouse\_table\_plus\_straight\_view**

Looker object: straight view (sql\_table\_name) over warehouse table

Why: M is a file extract with light Promote Headers / Changed Type. Looker best practice: load to warehouse, then use a normal view - not a derived table.

- \* 1. Land the CSV/Excel in cloud storage or ingest to a staging table.
- \* 2. Create curated table `YOUR\_PROJECT.YOUR\_DATASET.category` with casts matching M Changed Type.
- \* 3. Point generated LookML views/category.view.lkml sql\_table\_name at that table.
- \* 4. Do not use File.Contents paths from M in Looker.
- \* Files: m\_migration/sql/category.sql and lookml\_stubs/category\_recommended.lkml

### **film -> warehouse\_table\_plus\_straight\_view**

Looker object: straight view (sql\_table\_name) over warehouse table

Why: M is a file extract with light Promote Headers / Changed Type. Looker best practice: load to warehouse, then use a normal view - not a derived table.

- \* 1. Land the CSV/Excel in cloud storage or ingest to a staging table.
- \* 2. Create curated table `YOUR\_PROJECT.YOUR\_DATASET.film` with casts matching M Changed Type.
- \* 3. Point generated LookML views/film.view.lkml sql\_table\_name at that table.
- \* 4. Do not use File.Contents paths from M in Looker.
- \* Files: m\_migration/sql/film.sql and lookml\_stubs/film\_recommended.lkml

### **film\_actor -> warehouse\_table\_plus\_straight\_view**

Looker object: straight view (sql\_table\_name) over warehouse table

Why: M is a file extract with light Promote Headers / Changed Type. Looker best practice: load to warehouse, then use a normal view - not a derived table.

- \* 1. Land the CSV/Excel in cloud storage or ingest to a staging table.
- \* 2. Create curated table `YOUR\_PROJECT.YOUR\_DATASET.film\_actor` with casts matching M Changed Type.
- \* 3. Point generated LookML views/film\_actor.view.lkml sql\_table\_name at that table.
- \* 4. Do not use File.Contents paths from M in Looker.
- \* Files: m\_migration/sql/film\_actor.sql and lookml\_stubs/film\_actor\_recommended.lkml

### **film\_category -> warehouse\_table\_plus\_straight\_view**

Looker object: straight view (sql\_table\_name) over warehouse table

Why: M is a file extract with light Promote Headers / Changed Type. Looker best practice: load to warehouse, then use a normal view - not a derived table.

- \* 1. Land the CSV/Excel in cloud storage or ingest to a staging table.
- \* 2. Create curated table `YOUR\_PROJECT.YOUR\_DATASET.film\_category` with casts matching M Changed Type.
- \* 3. Point generated LookML views/film\_category.view.lkml sql\_table\_name at that table.
- \* 4. Do not use File.Contents paths from M in Looker.
- \* Files: m\_migration/sql/film\_category.sql and lookml\_stubs/film\_category\_recommended.lkml

### **film\_text -> warehouse\_table\_plus\_straight\_view**

Looker object: straight view (sql\_table\_name) over warehouse table

Why: M is a file extract with light Promote Headers / Changed Type. Looker best practice: load to warehouse, then use a normal view - not a derived table.

- \* 1. Land the CSV/Excel in cloud storage or ingest to a staging table.
- \* 2. Create curated table `YOUR\_PROJECT.YOUR\_DATASET.film\_text` with casts matching M Changed Type.
- \* 3. Point generated LookML views/film\_text.view.lkml sql\_table\_name at that table.
- \* 4. Do not use File.Contents paths from M in Looker.
- \* Files: m\_migration/sql/film\_text.sql and lookml\_stubs/film\_text\_recommended.lkml

### **inventory -> warehouse\_table\_plus\_straight\_view**

Looker object: straight view (sql\_table\_name) over warehouse table

Why: M is a file extract with light Promote Headers / Changed Type. Looker best practice: load to warehouse, then use a normal view - not a derived table.

- \* 1. Land the CSV/Excel in cloud storage or ingest to a staging table.
- \* 2. Create curated table `YOUR\_PROJECT.YOUR\_DATASET.inventory` with casts matching M Changed

Type.

\* 3. Point generated LookML views/inventory.view.lkml sql\_table\_name at that table.

\* 4. Do not use File.Contents paths from M in Looker.

\* Files: m\_migration/sql/inventory.sql and lookml\_stubs/inventory\_recommended.lkml

### **language -> warehouse\_table\_plus\_straight\_view**

Looker object: straight view (sql\_table\_name) over warehouse table

Why: M is a file extract with light Promote Headers / Changed Type. Looker best practice: load to warehouse, then use a normal view - not a derived table.

\* 1. Land the CSV/Excel in cloud storage or ingest to a staging table.

\* 2. Create curated table `YOUR\_PROJECT.YOUR\_DATASET.language` with casts matching M Changed Type.

\* 3. Point generated LookML views/language.view.lkml sql\_table\_name at that table.

\* 4. Do not use File.Contents paths from M in Looker.

\* Files: m\_migration/sql/language.sql and lookml\_stubs/language\_recommended.lkml

### **payment -> warehouse\_table\_plus\_straight\_view**

Looker object: straight view (sql\_table\_name) over warehouse table

Why: M is a file extract with light Promote Headers / Changed Type. Looker best practice: load to warehouse, then use a normal view - not a derived table.

\* 1. Land the CSV/Excel in cloud storage or ingest to a staging table.

\* 2. Create curated table `YOUR\_PROJECT.YOUR\_DATASET.payment` with casts matching M Changed Type.

\* 3. Point generated LookML views/payment.view.lkml sql\_table\_name at that table.

\* 4. Do not use File.Contents paths from M in Looker.

\* Files: m\_migration/sql/payment.sql and lookml\_stubs/payment\_recommended.lkml

### **rentat -> warehouse\_table\_plus\_straight\_view**

Looker object: straight view (sql\_table\_name) over warehouse table

Why: M is a file extract with light Promote Headers / Changed Type. Looker best practice: load to warehouse, then use a normal view - not a derived table.

\* 1. Land the CSV/Excel in cloud storage or ingest to a staging table.

\* 2. Create curated table `YOUR\_PROJECT.YOUR\_DATASET.rentat` with casts matching M Changed Type.

\* 3. Point generated LookML views/rentat.view.lkml sql\_table\_name at that table.

\* 4. Do not use File.Contents paths from M in Looker.

\* Files: m\_migration/sql/rentat.sql and lookml\_stubs/rentat\_recommended.lkml

### **staff -> warehouse\_table\_plus\_straight\_view**

Looker object: straight view (sql\_table\_name) over warehouse table

Why: M is a file extract with light Promote Headers / Changed Type. Looker best practice: load to warehouse, then use a normal view - not a derived table.

\* 1. Land the CSV/Excel in cloud storage or ingest to a staging table.

\* 2. Create curated table `YOUR\_PROJECT.YOUR\_DATASET.staff` with casts matching M Changed Type.

\* 3. Point generated LookML views/staff.view.lkml sql\_table\_name at that table.

\* 4. Do not use File.Contents paths from M in Looker.

\* Files: m\_migration/sql/staff.sql and lookml\_stubs/staff\_recommended.lkml

### **store -> warehouse\_table\_plus\_straight\_view**

Looker object: straight view (sql\_table\_name) over warehouse table

Why: M is a file extract with light Promote Headers / Changed Type. Looker best practice: load to warehouse, then use a normal view - not a derived table.

\* 1. Land the CSV/Excel in cloud storage or ingest to a staging table.

\* 2. Create curated table `YOUR\_PROJECT.YOUR\_DATASET.store` with casts matching M Changed Type.

\* 3. Point generated LookML views/store.view.lkml sql\_table\_name at that table.

\* 4. Do not use File.Contents paths from M in Looker.

\* Files: m\_migration/sql/store.sql and lookml\_stubs/store\_recommended.lkml

## **customer -> warehouse\_table\_plus\_straight\_view**

Looker object: straight view (sql\_table\_name) over warehouse table

Why: M is a file extract with light Promote Headers / Changed Type. Looker best practice: load to warehouse, then use a normal view - not a derived table.

- \* 1. Land the CSV/Excel in cloud storage or ingest to a staging table.
- \* 2. Create curated table `YOUR\_PROJECT.YOUR\_DATASET.customer` with casts matching M Changed Type.
- \* 3. Point generated LookML views/customer.view.lkml sql\_table\_name at that table.
- \* 4. Do not use File.Contents paths from M in Looker.
- \* Files: m\_migration/sql/customer.sql and lookml\_stubs/customer\_recommended.lkml

## 6. Gaps - prioritize and close

Each gap blocks production readiness until resolved or formally accepted.

### Severity HIGH (4)

#### GAP [HIGH] - Connection & warehouse

What is missing: LookML still uses placeholders YOUR\_LOOKER\_CONNECTION and YOUR\_PROJECT.YOUR\_DATASET.

##### What to do:

Set the real Looker connection name and point every sql\_table\_name at existing warehouse tables before validating.

#### GAP [HIGH] - Power Query / ETL

What is missing: 16 Power Query queries need warehouse/Looker equivalents. Pattern mix: {warehouse\_table\_plus\_straight\_view': 16}.

##### What to do:

Open LOOKML\_PROJECT.zip -> lookml/m\_migration/. For each query follow M\_QUERY\_RECOMMENDATIONS.md, implement sql/<query>.sql in the warehouse, then update the straight LookML view sql\_table\_name. Use LookML SDT only when the recommendation allows a temporary bridge.

#### GAP [HIGH] - Complex DAX

What is missing: 2 of 5 measures are TODO stubs (CALCULATE/time-intel/iterators).

##### What to do:

Implement each TODO using LookML filters, period-over-period patterns, or warehouse logic. Keep original DAX in the field description until KPI parity passes.

#### GAP [HIGH] - Calculated columns

What is missing: 1 business calculated columns must be materialized (prefer warehouse), not left as DAX.

##### What to do:

Create warehouse columns for: customer.Employment Duration

### Severity MEDIUM (1)

#### GAP [MEDIUM] - Inactive relationship

What is missing: Inactive in Power BI: rentat[staff\_id] -> staff[staff\_id] (M:1).

##### What to do:

Looker has no inactive join. Keep as a separate aliased join (from:.) and only expose when a measure needs USERRELATIONSHIP-style behavior. Hide fields until needed.

### Severity LOW (2)

#### GAP [LOW] - Row-level security

What is missing: No RLS roles in this PBIX inventory.

##### What to do:

Still confirm with security owners whether Looker needs access\_filter by region/org.

#### GAP [LOW] - Auto date tables

What is missing: 20 LocalDateTable\_/DateTableTemplate\_ tables skipped (correct).

##### What to do:

Use business date columns with dimension\_group timeframes. Do not migrate auto-date tables.

## 7. What to check (validation)

### LookML compile

- \* ☐ Validator: 0 errors
- \* ☐ One primary\_key per view
- \* ☐ Every join has relationship + sql\_on
- \* ☐ Dialect-correct SQL

### Grain and joins

- \* ☐ Fact row count matches Power BI (same filters)
- \* ☐ count vs count\_distinct(PK) shows no unexpected fan-out
- \* ☐ Orphan FK rate understood for left\_outer joins

### Business logic

- \* ☐ All HIGH gaps closed or signed off
- \* ☐ Calculated columns available
- \* ☐ Required measures implemented and spot-checked vs Power BI
- \* ☐ Inactive / M:M joins reviewed

### Security and ops

- \* ☐ RLS / access\_filter decision recorded
- \* ☐ Caching / datagroup set
- \* ☐ QA explores hidden from end users

## 8. Acceptance criteria

- \* Validator clean on target connection
- \* HIGH gaps closed or formally accepted
- \* At least 5 Power BI business questions reproduce in Looker within agreed tolerance
- \* Remaining MEDIUM/LOW gaps have owners and dates
- \* Do NOT publish claiming full KPI parity while DAX TODOs remain

## 9. Inventory snapshot

- \* tables: 36
- \* business\_tables: 16
- \* internal\_tables: 20
- \* columns: 231
- \* measures: 5
- \* calculated\_columns: 121
- \* calculated\_tables: 20
- \* relationships: 16
- \* power\_query: 16
- \* m\_files: 16
- \* rls\_roles: 0
- \* hierarchies: 20
- \* partitions: 322
- \* auto\_date\_tables: 20
- \* annotations: 433
- \* sort\_by\_columns: 0
- \* format\_strings: 63
- \* perspectives: 0
- \* display\_folders: 0

## 10. Out of scope

- \* Power BI report pages, visuals, bookmarks, themes
- \* Automatic warehouse DDL generation
- \* Automatic KPI certification

Generated by phase2/generate\_developer\_guide.py for movie\_rental\_analysis.pbix.